Today

Welcome to your next lesson: Polymorphism basics: one action, different behaviors.

I'm getting your personalized lesson ready—based on what you've learned, what's coming up, and the best way to help it all click.

Let's keep moving forward, step by step.

Imagine a pet-care app 🐾

You have different pets: a generic pet, a dog, and a cat.

You want to tell each one to do the same action: speak().

Dog should go: "Woof!"
Cat should go: "Meow!"
Generic pet might go: "...some pet sound..."

The command (speak()) is the same, but the behavior depends on which object you call it on.

That "one action, different behaviors" idea is what we call polymorphism.

Gut check for you:

If you see this in Python:

Python
dog.speak()
cat.speak()
Do you expect these two lines to:

feel like “the same action being used in two places”, or feel like “two totally different actions”?

Just answer in your own words, then we’ll build a tiny Dog/Cat example together.

Same action, different behavior

Exactly 🎯 — same action name, different behavior based on the object. That’s the core of polymorphism.

Let’s make it concrete with a tiny Python example.

Python

```
class Pet:
    def speak(self):
        print("Some pet sound")

class Dog(Pet):
    def speak(self):
        print("Woof!")

class Cat(Pet):
    def speak(self):
        print("Meow!")

pet = Pet()
dog = Dog()
cat = Cat()

pet.speak()
dog.speak()
cat.speak()
```

Don’t run this yet — just think:

What do you expect to be printed by these three lines, in order?

You can just describe it like:

“line 1: …, line 2: …, line 3: …”

I’m not sure yet

No worries at all—that’s what we’re here for 😊

Let’s walk it very slowly, one line at a time.

Here’s the code again, just the key part:

Python

```
pet = Pet()
dog = Dog()
cat = Cat()

pet.speak()
dog.speak()
cat.speak()
```

And up above we had:

Python

```
class Pet:
    def speak(self):
        print("Some pet sound")

class Dog(Pet):
    def speak(self):
        print("Woof!")

class Cat(Pet):
    def speak(self):
        print("Meow!")
```

Think step by step:

pet.speak() → uses the Pet version of speak() → prints: Some pet sound
dog.speak() → uses the Dog version → prints: Woof!
cat.speak() → uses the Cat version → prints: Meow!

So the output would be:

Text

```
Some pet sound
Woof!
Meow!
```

Same method name (speak()), different behavior depending on the object’s class—that’s polymorphism in action 🐶🐱

Does this step-by-step mapping from each object → its speak() → its line of output make sense so far?

If yes, I'll have you do a tiny variation yourself next.

ididnt hit i dont know it was a mosue glitch
Some pet sound
Woof!
Meow!

Got it, thanks for saying that 🙌 And you did get the outputs right:

Text
Some pet sound
Woof!
Meow!
So you already matched:

pet.speak() → Some pet sound
dog.speak() → Woof!
cat.speak() → Meow!

That's exactly how polymorphism behaves here: same call (.speak()), class decides what actually runs.

Quick check-in before we do a tiny hands-on step:

Do you feel clear on why dog.speak() doesn't use the Pet version, but the Dog version instead?

You can answer:

"Yes, because overriding"
"Kind of, but I want it explained again"
or tell me where it feels fuzzy.
yes same method name overriding the parent version

Nice, exactly 💪 — overriding the parent version is the engine behind polymorphism.

Let's zoom in on one idea now: a polymorphic call.

A polymorphic call is when you write the same method call, like speak(),

and let the object’s class decide which version runs.

In our example:

Call: pet.speak() → class: Pet → runs Pet.speak
Call: dog.speak() → class: Dog → runs Dog.speak
Call: cat.speak() → class: Cat → runs Cat.speak

The call text (something.speak()) looks the same every time.

What changes is what something actually is.

Make sure this part is solid before we move on:

If I write animal = Dog() and then call animal.speak(),

which implementation runs: Pet.speak or Dog.speak? 🐕

Just answer in one word: Pet or Dog.

Dog.speak

Perfect 🎯

animal = Dog() then animal.speak() → runs Dog.speak. The object’s class wins.

That’s the heart of polymorphism:

the call is the same, the class behind the object picks the behavior.

Let’s do a tiny variation to make you drive a bit 🚗

I want you to use the code editor for this (not the chat):

Start from this version:

```
Python
class Pet:
    def speak(self):
        print("Some pet sound")

class Dog(Pet):
    def speak(self):
        print("Woof!")

class Cat(Pet):
    def speak(self):
        print("Meow!")

pet = Pet()
dog = Dog()
cat = Cat()

pet.speak()
dog.speak()
cat.speak()
```

In the editor, change the Pet.speak method so it prints Silent pet instead of Some pet sound.
Run it, then Submit code.

Don’t tell me the output yet—just make the change, run, and submit. Then I’ll ask you what changed and why.

```
Python
class Pet:
    def speak(self):
        print("Silent pet")

class Dog(Pet):
    def speak(self):
        print("Woof!")

class Cat(Pet):
    def speak(self):
        print("Meow!")

pet = Pet()
dog = Dog()
cat = Cat()
```

```
pet.speak()
dog.speak()
cat.speak()
```

Nice work, Ron 🎯 Your change did exactly what I hoped.

Your output:

```
Text
Silent pet
Woof!
Meow!
```

Step by step:

pet.speak() → uses Pet.speak → now prints Silent pet
dog.speak() → still uses Dog.speak → Woof!
cat.speak() → still uses Cat.speak → Meow!

Notice: you changed only the parent method, and only the parent object's behavior changed. The overridden versions in Dog and Cat stayed in control for those objects. That's overriding + polymorphism working together.

Quick check for understanding:

If you now changed Dog.speak to print Bark bark!, what exactly would you expect the three lines of output to be, in order?

Just write the three lines as you expect them.

```
Silent pet
Bark bark!
Meow!
```

Exactly right ✅

If you changed Dog.speak to:

```
Python
class Dog(Pet):
    def speak(self):
```

```
        print("Bark bark!")
then the output would be:

Text
Silent pet
Bark bark!
Meow!
Still Silent pet for pet.speak() (from Pet)
Now Bark bark! for dog.speak() (from Dog)
Still Meow! for cat.speak() (from Cat)

Same call each time (something.speak()), behavior decided by the object’s class. That’s polymorphism in action 🌀

Let’s step outside pets for a moment to see if the pattern transfers.


Imagine:

Python
class Shape:
    def describe(self):
        print("I am a shape")

class Circle(Shape):
    def describe(self):
        print("I am a circle")

class Square(Shape):
    def describe(self):
        print("I am a square")

shape = Shape()
circle = Circle()
square = Square()

shape.describe()
circle.describe()
square.describe()
You don’t need to run this—just think:


What shared action name is being used polymorphically here (like speak() was for pets)?

You can answer with just the method name.
```

describe

Yes 🙌 the shared polymorphic action here is describe().

shape.describe() → "I am a shape"
circle.describe() → "I am a circle"
square.describe() → "I am a square"

Same call name: .describe(), different behavior per class — exact same pattern as speak() with pets.

Now, big picture in simple words:

Polymorphism = I write code that just says “call speak()” or “call describe()”,

and I don’t need different method names for Dog/Cat or Circle/Square.

The object’s class decides what actually happens.

This makes program logic simpler: one shared interface (speak, describe), many behaviors.

Before we do a quick final check:

Does this idea of “one shared method name, objects supply their own behavior” feel clear?

Reply “yes” or “almost but…” and then I’ll give you a tiny final question.

Yes, it's clear

What do you think the right answer is here? 👇

A Dog and a Cat both have a speak() method. What is the polymorphic superpower?

A.
They must both make the same sound.

B.
The same method call can produce different behavior for each object.

C.
Only the parent class can call speak().

D.
The objects stop being instances of their classes.

You nailed it 🎉

Polymorphic superpower =

“The same method call can produce different behavior for each object.”

In our case:

dog.speak() → Woof!
cat.speak() → Meow!

The call is the same (.speak()), but the class of the object (Dog vs Cat) supplies a different implementation. That’s exactly what polymorphism is about: one shared action name, many behaviors.

You’ve shown you can:

Predict which implementation runs based on the object’s class ✅
See how overriding powers polymorphism ✅
Recognize that this simplifies program logic (one method name, multiple types) ✅

You’ve mastered this lesson’s core ideas 💪

Do you feel ready to wrap up this polymorphism lesson and move on to the next one, or would you like one more quick practice example first?

Ready to move on

Nice work, Ron 🎯

Here’s what you’re taking with you from this lesson:

Polymorphism: one shared action name (speak(), describe()) with different behaviors depending on the object.
You saw that the method call stays the same, but the object’s class (Dog vs Cat, Circle vs Square) decides which overridden method runs.
You used this to simplify logic: instead of dog_bark(), cat_meow(), etc., you can just call speak() on any pet and trust its class to do the right thing.

You answered the final question perfectly and wrote/modified code that used polymorphism correctly — that’s solid mastery 💪

When you’re ready, go ahead and hit Next Lesson in the Learn tab, and we’ll build on this foundation step by step.

```
class Pet:
    def speak(self):
        print("Silent pet")

class Dog(Pet):
    def speak(self):
        print("Woof!")

class Cat(Pet):
    def speak(self):
        print("Meow!")

pet = Pet()
dog = Dog()
cat = Cat()
```

```
466
467   pet.speak()
468   dog.speak()
469   cat.speak()
```